



Önálló laboratórium beszámoló

Távközlési és Mesterséges Intelligencia Tanszék

Név: **Orbán Levente**

orban.levente.laszlo@gmail.com

Neptun-kód: **CZ6NDB**

Szakirány: **Szoftverfejlesztés**

Konzulens: **Németh Gábor**

nemethgab@tmit.bme.hu

Téma címe: Interaktív SKI-kombinátor kalkulus fordító és vizualizáló

Feladat:

A hallgató feladata egy webes eszköz (Combinator Explorer) elkészítése, amely SKI-kifejezések esetén megmutatja a redukciós lépéseket, vizualizálja a redexeket, és felrajzolja a kiértékelési fát. A redukció során többféle kiértékelési stratégiát támogat, valamint az SKI-kifejezéseket lefordítja λ -kalkulusba. Az eszköz hidat képez a formális elmélet és a gyakorlati megvalósítás között: oktatási segédeszköz, amely mélyíti a funkcionális programozás, illetve a fordítóprogramok működésének megértését.

Tanév: 2025/26. tanév, II. félév

1. A laboratóriumi munka környezetének ismertetése, a munka előzményei és kiindulási állapota

1.1. Bevezető

Az elmúlt három évtizedben egyre nagyobb jelentőségre tett szert a funkcionális programozás az iteratív és objektumorientált paradigmákkal szemben. Az figyelhető meg, hogy azok a programozási nyelvek, amelyek tradicionálisan objektumorientált szemléletűek voltak (például C++, Java) napjainkra jelentős számú funkcionális elemmel bővültek (anonim függvények, lista komprehenzió, első osztályú függvények, stb.). Továbbá azok a nyelvek, amelyek az elmúlt évtizedben tettek szert népszerűsége (például Rust) már a kezdetektől fogva egy több paradigmát ötvöző nézettel lettek tervezve. Ezeket az új eszközöket viszont – mind a már piacon lévő mérnökök továbbképzése, mind a felsőoktatásban résztvevők oktatása szempontjából – pedagógiaiilag még a mai napig nehéz kielégítően oktatni. A funkcionális paradigma – illetve annak a matematikai alapjai – idegenek lehetnek a mérnöki beállítottságú közönség számára.

A jelen írásos beszámolómban bemutatott szoftver ezt a pedagógiai hiányt próbálja pótolni: egy olyan vizuális, interaktív, webes eszközt ad a pedagógusok kezébe, amellyel a funkcionális programozást egészen a matematikai alapokig visszamenően lehet hatékonyabban oktatni, hozzá pontosabb intuíciót építeni.

A választott megközelítés (vizuális, webes program) nem egyedi. Számtalan más olyan témakörhöz, amelyek tradicionálisan pedagógiai kihívásokat jelentettek (generált gépi kód, vagy reguláris kifejezések elemzése) készült hasonló alapötletekkel rendelkező szoftver (mint például a [Compiler Explorer](#), vagy [Reg-ex101](#)).

A jelen beszámolómban bemutatott megközelítés a modern funkcionális nyelvek matematikai alapjait kívánja megcélolni (többek között redukciós láncok és gráfok, kiértékelési sorrendek, λ - és SKI-kalkulus). Ezen alaptudás ismeretében a legtöbb modern funkcionális nyelv (ML-család, Lisp-család) szemantikáinak a működése – legalábbis magas szinten – könnyebben elsajátítható.

1.2. Elméleti összefoglaló

1.2.1. λ -kalkulus

A modern funkcionális programozási nyelvek alapjaikban a (típus nélküli) λ -kalkulusra építenek¹, amely egy általános célú, Turing-teljes programozási nyelv. Az egyik legegyszerűbb Turing-teljes nyelv, hiszen csak három konstrukcióra épít (lambda-kifejezések, lambda-absztrakciók, és applikációk). A λ -kalkulus nyelvtanának formális definíciója az 1. ábrán látható.

$$\begin{aligned} \langle \lambda\text{-kifejezés} \rangle & ::= \langle \text{változó} \rangle \\ & \quad | \langle \lambda\text{-absztrakció} \rangle \\ & \quad | \langle \text{applikáció} \rangle \\ \langle \lambda\text{-absztrakció} \rangle & ::= ' \lambda ' \langle \text{változó} \rangle ' . ' \langle \lambda\text{-kifejezés} \rangle \\ \langle \text{applikáció} \rangle & ::= \langle \lambda\text{-kifejezés} \rangle \langle \lambda\text{-kifejezés} \rangle \end{aligned}$$

1. ábra. A λ -kalkulus szintaktikája

A λ -absztrakció segítségével tudunk függvényeket definiálni, amelyeknek paramétereiket az applikáció konstrukciójával tudunk adni. Ha kicsit módosítjuk a nyelvtanunkat, hogy szám konstansokat, és egyéb kényelmes konstrukciókat megengedjen, akkor írhatjuk például:

$$\begin{aligned} & \lambda x. (5 * x) \\ & (\lambda x. (5 * x))(3) \\ & 15 \end{aligned}$$

¹Tekinthetők a λ -kalkulus kibővített, változatának (legyen szó típusokról, vagy let-kötésekről, etc.). Példa erre az ML-család (Standard ML, Caml, Haskell), amelyek közelítőleg felfoghatóak úgy, mint a λ -kalkulus polimorf Hindley-Milner típusrendszerrel való kiegészítése.

Az első sorban definiálunk egy függvényt, amely ötszörözi a bemenetét. A második sorban meghívjuk ezt a függvényt a 3 bemenettel. Az utolsó sorban a szorzás eredménye látható. Fontos megjegyezni, hogy az applikáció bal-asszociatív.

1.2.2. A SKI-kalkulus

A kombinatorok absztrakt matematikai konstrukciók, viszont megértésük és történelmi hátterük kritikus fontosságú a modern számítástudomány, különösen a funkcionális programozás, vagy a kiszámíthatóság modelljeinek vizsgálatához.

Általánosságban különböző problémák formális, szimbolikus ábrázolását már az ókorban megkísérelték. Így született Arisztotelész logikája és Euklidész geometriája. A számítástudomány szempontjából viszont a 19. században a Boole-algebra megjelenése jelentette a legnagyobb áttörést. A 20. század fordulóján a matematikusok arra tettek kísérletet, hogy a teljes matematikát egy szigorú, tiszta logikai alapra helyezték. Ahogy ezek a logikai keretrendszerek egyre nagyobbak lettek, megjelent az igény a bizonyos axiómák meghatározására. Ennek a törekvésnek a legismertebb példája a Sheffer-vonás felfedezése, amely bebizonyította, hogy a teljes ítéletkalkulus felépíthető egyetlen logikai kapu segítségével (NAND). Felmerült a kérdés: ha a logikai axiómák ilyen elemi szintre redukálhatóak, megtehető-e ugyanez általánosságban a függvényekkel és a változókkal is?

Moses Schönfinkel 1920-ban pontosan erre tett kísérletet [5]. Felismerte, hogy ha a függvényeket kellőképpen általánosítjuk (egy függvény egy másik függvényt is kaphat paraméterként, és visszatérési értéke is lehet egy függvény), a változók feleslegessé válnak. Ezzel az elképzeléssel lefektette a kombinatorkalkulus alapjait. A kombinatorkalkulus leghíresebb és leginkább letisztult² formalizációja az SKI-kalkulus, amelyet Schönfinkel munkássága nyomán Haskell Curry dolgozott ki [2]. Ez pusztán három előre definiált kombinator egymás utáni alkalmazásával is képes Turing-teljes számítások leírására.

A rendszer a következő három kombinatorból áll:

- Az egyváltozós identitás kombinator (I), amely beavatkozás nélkül, egyszerűen visszaadja a kapott argumentumát.
- A kétváltozós konstans kombinator (K), amely olyan függvényt állít elő, ami a második argumentumát mindig figyelmen kívül hagyja, és visszaadja az első.
- A háromváltozós szubsztitúciós kombinator (S), amely az argumentumok szétosztását és a függvényalkalmazás láncolatát végzi.

Bár a rendszert általában további kombinatorokkal is ki szokás egészíteni a kényelmesség és átláthatóság érdekében, ezek mind levezethetők a fenti három kombinatorból.

Ha a λ -kalkulus, és a SKI-kalkulus és Turing-teljesek, akkor bizonyára kell lennie valamilyen ekvivalenciának közöttük. Alább megadom a SKI-kalkulus három kombinatorának a λ -kalkulus szerinti definícióját:

$$\begin{aligned} I &= \lambda x.x \\ K &= \lambda x.(\lambda y.x) \\ S &= \lambda x.(\lambda y.(\lambda z.xz(yz))) \end{aligned}$$

Ezeknek a szabályoknak az alkalmazásával tetszőleges SKI-kalkulus kifejezést át lehet alakítani λ -kalkulus kifejezéssé, melyre a programban lehetőség is van.

1.3. A munka állapota, készültségi foka a félév elején

A jelen félév előtti szünetben már elkezdtem feldolgozni a témához releváns irodalmat. A 2025/26/I. félévben egy olyan tárgyat hallgattam, amelyben jelentős szerepe volt a λ -kalkulusnak, kategória elméletnek. Ezek az előadások nagyban hozzájárultak az előzetes tudás megszerzéséhez. A konzulensem számos releváns könyvet ajánlott feldolgozásra, többek között Smullyan [6] és Wolfram [7, 8] írásait.

²Bár a fent említett SKI rendszer az általánosan vizsgált forma, fontos megemlíteni, hogy léteznek kevesebb kombinatorokat igénybe vevő, Turing-teljes rendszerek is. Ilyen például az S és K kombinatorokból álló bázis. Ez abból adódik, hogy az I kombinator kifejezhető a másik kettőből, például (SKK) módon.

2. Az elvégzett munka és az eredmények ismertetése

2.1. Az architektúra megtervezése

Az architektúra tervezésének folyamatában fordítóprogramokból merítettem inspirációt, különös tekintettel a moduláris felépítésre és az úgynevezett *Separation of Concerns* elvére.

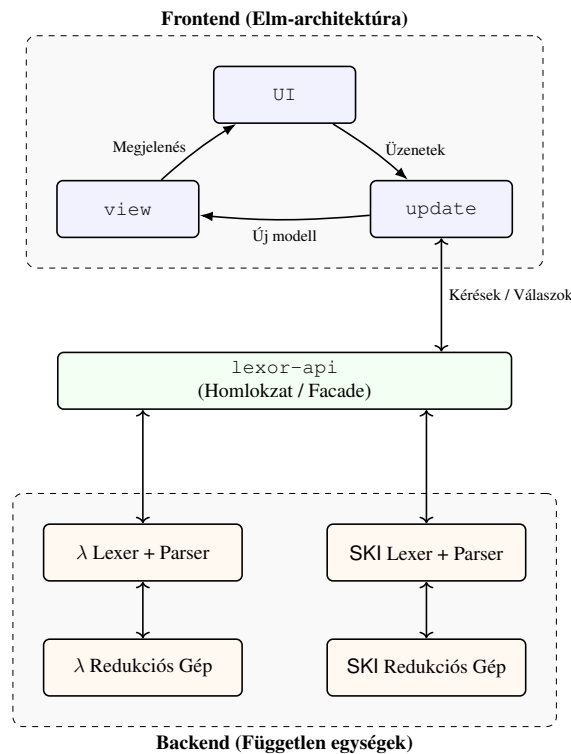
A nyelvtanok egyszerűsége miatt a lexert és a parsert egybevettem. A parserek megírásához a **chumsky** könyvtárat használtam, amely egy parser combinator alapú módszert használ, rekurzív leszállást igénybe véve. Két redukciós gépet implementáltam: egyet ami λ -kalkuluson, egyet ami SKI-kalkuluson operál. A két parser és a hozzájuk tartozó redukciós gépekre egy egységként tekintettem, ezek alkották a backend könyvtárat.

A frontend és a backend közötti tiszta kommunikáció megteremtése érdekében a Homlokzat (Facade) programtervezési mintát alkalmaztam. Ez az API réteg (`lexor-api`) elrejt a redukciós gépek komplexitását, és egy egységes interfészt biztosít a webes felület számára.

A frontend terén az úgynevezett *Elm-architektúrát* követtem, amely egy üzenetsor-alapú architektúra. Ez a megközelítés három részre bontja a frontendet:

- A kezdeti `model`-re, amely az applikáció állapotát írja le. Ez az állapot nem mutábilis, tehát ahogy a felhasználó interakcióba lép vele, mindig egy teljesen új állapot jön létre.
- A `view` függvényre, amely a modell megjelenését írja le. A felhasználói interakciókat itt alakítom át üzenetekké.
- Az `update` függvényre, amely az üzenetsort kiolvastva végrehajtja a felhasználó összes parancsát a jelenlegi modellen, ezáltal egy új modellt létrehozva.

Ahogy a felhasználó interakcióba lép a felhasználói felülettel, úgy új üzenetek kerülnek az üzenetsorba. Az üzenetsort minden frame végén az `update` függvény kiüríti, és a tartalmát végrehajtja. Ezáltal egy új modell jön létre, amit átad a `view` függvénynek, ami egy új UI-t hoz létre. Az architektúra egyszerűsített vizuális modellje a 2. ábrán látható.



2. ábra. Az alkalmazás rétegei és architektúrája

Mind a frontend, mind a backend WebAssembly modulokra kerül fordításra, tehát az egész program a felhasználó böngészőjében tud futni kliensoldalon. Annak érdekében, hogy a CPU igényes backend számítások alatt a frontend ne fagyjon be, egy külön háttérzálon futnak a backend számításai, amelyhez Web Workereket használók.

2.2. A redukciós gépek implementálása

2.2.1. A SKI-kalkulus

A redukciós gépek implementálásához a Graph Reduction Machine (G-machine) koncepcióját vizsgáltam meg. Optimalizálási célból a fa-alapú kiértékelés mellett az alkifejezések megosztása mellett döntöttem, amellyel exponenciálisan kevesebb lépésszámot vesznek igénybe az egyes redukciók [4]. Az elterjedtebb "sztringredukció" – azaz amikor a bemenetet szövegmanipulációs módszerekkel alakítjuk át a redukciós szabályoknak megfelelően – nem skálázódik elég jól hosszabb kifejezések kiértékelésére, illetve az előbb említett alkifejezés-megosztás technikáját sem tudom érvényesíteni.

A követelmények elemzése és a belső reprezentáció kialakítása után implementáltam egy G-machine-t mind a λ -kalkulus, mind a SKI-kalkulus esetében. Az ilyen redukciós gépek implementálása során gyakran használt, fa alapú AST (Abstract Syntax Tree, absztrakt szintaxis fa) sokszor memóriatöredezettséghez és lassabb futáshoz vezethet. Ennek elkerülése érdekében a Flat-AST (laposított AST) reprezentációs technikát használtam, amelyet egy aréna alapú memória-allokátorral párosítottam [3]. Egy rekurzív, fa alapú megközelítés lehet az 1. listingben bemutatott modell. Itt pointereket tárolunk el a többi csúcra, amely

```
1 enum NodeRec {
2     S, K, I,
3     App(Box<Self>, Box<Self>),
4     Indirection(Box<Self>),
5 }
```

1. Listing. Egy rekurzív, mutató alapú gráf struktúra

csökkentheti a memória lokalitását, illetve az adatstruktúra méretét is növelheti 64-bites rendszereken. Ehhez képest az aréna-allokátoros megközelítés – melyet a 2. listing mutat be – nem használ mutatókat. Minden csúcra az indexe alapján lehet hivatkozni, mely alapján az arénából kinyerhető. Így a különböző kifejezések akár teljesen szomszédos memóriarégiókban lehetnek, növelve a cache-kihasználtságot.

```
1 enum NodeFlat {
2     S, K, I,
3     App(NodeIdx, NodeIdx),
4     Indirection(NodeIdx),
5 }
```

2. Listing. Egy aréna-allokátoros gráf struktúra

A G-machine a kombinátorok redukcióját egy állapotgép segítségével végzi. A kiértékelés során először végighaladunk a kifejezés "gerincén", amely azokból a csúcsokból áll, amit úgy kapunk, ha a fa gyökerétől kezdve addig megyünk balra, ameddig van baloldali gyerek. A gerincen összegyűlt csúcsok argumentumait a redukciós gép a megfelelő kombinátor szabályai szerint helyettesíti be (például az S, K, I kombinátorok esetében). A redukciós szabályok alkalmazását végrehajtó Rust kódrészlet a 3.2. szakaszban, a csatlakozó dokumentumok között tekinthető meg.

Mivel a szoftver célja az oktatás megkönnyítése, több redukciós stratégiát is támogatnom kellett (például Normal order, Call-by-value). Ezek megértéséhez elemezzük a 3. ábrát [7], amely az alábbi kombinátor AST-ét ábrázolja:

$$S(S(S)(S)(S(K(K)(S))(S)))(S)(S)(S(K(S)(K))(K)(S))$$

Az ábrán látható fában a szürke belső csomópontok az applikációkat (függvényalkalmazásokat) jelölik, a levelek pedig az egyes kombinátorokat. Látható, hogy egy ilyen komplex kifejezés esetén egyszerre több

ponton is elvégezhető lenne a redukció. A támogatott stratégiák pontosan abban térnek el, hogy ezek közül melyiket választják:

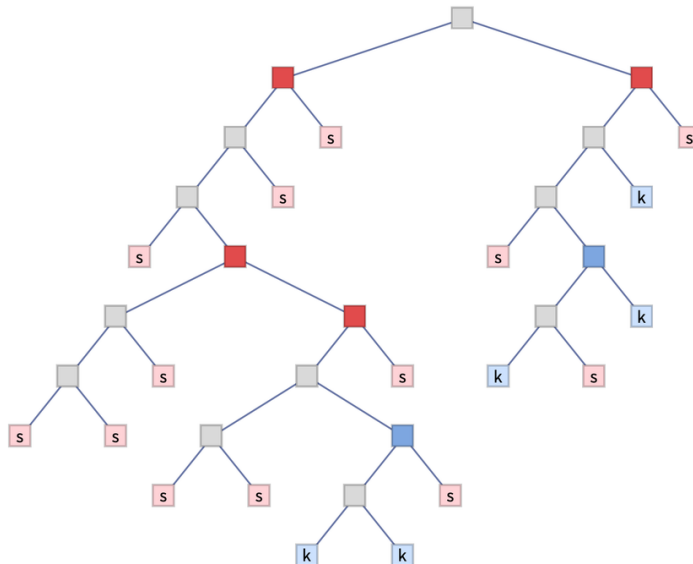
- Normál sorrend (Normal order és Call-by-name): Mindig a legbaloldalibb, legkülső (leftmost-outermost) redexet választják ki. A gráfban ez a gyökértől balra haladva a legmagasabban fekvő kiemelt csomópont. A két stratégia között az a fő különbség, hogy míg a Normal order egy nem applikált függvény törzsén belül is elvégzi a redukciós lépéseket, addig a Call-by-name nem tesz ilyet, hanem megáll amint elér egy telítetlen absztrakciót (amelynek nincsen elég argumentuma ahhoz, hogy kiértékeljük). Ez utóbbi emiatt az úgynevezett Weak head normal form állapotba hozza a kifejezéseket, ahol nincsen arra garancia, hogy az argumentumok között még nincsen kiértékelhető alkifejezés. A Normal order esetén normálformába kerülnek a kifejezések, amit már nem lehet tovább egyszerűsíteni. Ez a két stratégia a redukálást a 3. ábrán a $S(S(S)(S)(S(S)(K(K)(S)))(S)))(S)(S)$ alkifejezéssel kezdené.
- Applikatív sorrend (Applicative order és Call-by-value): A legbaloldalibb, legbelső (leftmost-innermost) redexet redukálják először. Hasonló különbség létezik ezen két stratégia között is: a Call-by-value nem végez kiértékelést az absztrakciók törzsén belül, míg az Applicative order igen. A gyakorlatban ezek abban térnek el az előző pontban említett stratégiáktól, hogy egy függvény alkalmazása előtt – amennyiben annak minden paramétere rendelkezésre áll – az argumentumait értékeli ki, a fában lentől felfelé haladva. Ez a két stratégia a kiértékelést a 3. ábrán a $K(K)(S)$ alkifejezéssel kezdené. A legtöbb modern imperatív és szigorú nyelv (pl. C++, Rust, OCaml) ezt használja.

A stratégiák oktatása azért kiemelten fontos, mert vannak olyan SKI-kalkulus kifejezések, amelyeknek a redukciója bizonyos stratégiákat alkalmazva végtelen ciklusba kerül. Tekintsük például az alábbi kifejezést:

$$KI((SII)(SII))$$

Applikatív sorrend alkalmazásával, a gép először megpróbálja az argumentumokat $((SII)(SII))$ teljesen kiértékelni, mielőtt a K kombinátort alkalmazná. Ez egy végtelen ciklust eredményez. Normál sorrend szerint viszont az algoritmus a legkülső redexet (a K applikációját) választja ki először, ezáltal a második argumentum (a divergáló kifejezés) eltűnik, és a kifejezés normál-formába kerül (I).

A kialakításban különösen figyeltem arra, hogy a redukciós logika moduláris maradjon, így az egyes stratégiák könnyen cserélhetők és összehasonlíthatók anélkül, hogy a motor alapvető működését módosítani kellene.



3. ábra. Egy kombinátor absztrakt szintaxis fája. A különböző kiértékelési módok kiválóan bemutatathatóak rajta.

2.2.2. A λ -kalkulus

A λ -kalkulus kiértékelése bizonyos szempontból bonyolultabb, mint a SKI-kalkulusé. Ez részben abból adódik, hogy a λ -kalkulusban vannak változók. Létezik viszont a λ -kalkulusnak egy olyan alakja, amely – bár teljesen nem tud megszabadulni tőlük – egyszerűbbé teszi a kezelésüket. Ez az alak, a de Bruijn forma³[1]. Ez a forma az úgynevezett de Bruijn indexeken alapul: egy λ -kalkulus kifejezésben egy változó de Bruijn indexe az a szám, amely azt jelzi, hogy az őt "kötő" λ -absztrakció hány szinttel van feljebb a kifejezés fájában. Például a $\lambda x.(\lambda y.x)$ kifejezésben az x változó de Bruijn indexe 2, mivel az őt definiáló λ -absztrakció ($\lambda x.$) az két szinttel van feljebb. A kifejezés de Bruijn alakja tehát $\lambda \lambda 2$.

Az én λ -kalkulus kiértékelőm ilyen alakban végzi el a kiértékelést, mivel így a kiértékelés során nem kell külön figyelni az úgynevezett α -ekvivalenciára. Ehhez meg kellett valósítanom a konverziót a tradicionális λ -kalkulus és a de Bruijn forma között mindkét irányban. Ezek után a de Bruijn alakra definiáltam a β -redukciót, amely megfelel egy λ -absztrakció kiértékelésének egy változóval. Ez után definiáltam a különböző stratégiákat, amelyek csak abban térnek el, hogy a kifejezésben éppen melyik alkifejezést β -redukálják.

Három stratégiát valósítottam meg: Normal order, Call-by-name illetve Call-by-value. Azért erre a háromra esett a választás, mert szerettem volna mind lazy és eager, mind normálformára és nem-normálformára hozó stratégiákat elérhetővé tenni.

2.3. A tesztelő órakulumban kialakítása

A szoftver helyességének verifikálására a hagyományos egységtesztek mellett a Property-based testing (tulajdonság-alapú tesztelés) módszertanát alkalmaztam. Ennek során a teszt-keretrendszer automatikusan generál valid, véletlenszerű λ - és SKI-kifejezéseket, majd megvizsgálja a rendszer invariáns tulajdonságait.

A redukciós gépek kimenetének validálásához egy tesztelő órakulumban alakítottam ki. Az órakulumban egy már bizonyítottan helyesen működő, külső interpreter, amelyet egy elszigetelt Podman konténerben futtattam le a biztonság és a determinisztikus környezet garantálása érdekében. A tesztelési folyamat során a `lexor` és a Podman konténerben futó órakulumban is kiértékelte ugyanazt a generált kifejezést, a teszt-keretrendszer pedig a két kimenet ekvivalenciáját vizsgálta Black-box (fekete doboz) tesztelés formájában.

³Az általam használt változata a lokálisan névtelen de Bruijn forma

2.4. A vizuális redukciós elemek létrehozása

Egy oktatási eszköz esetében a belső folyamatok láthatósága kritikus jelentőségű, ezért a backendet felkészítettem arra, hogy úgynevezett *callback* függvényeket tudjon fogadni. Ezek a függvények egy olyan nézetet kapnak a belső gépről, amelyen olyan metódusok vannak definiálva, amelyekkel a teljes AST-t be lehet járni, szemantikai információt fűzve az egyes csúcsokhoz. Így az egyes redukciós lépések könnyen naplózhatóak, az alkifejezések különböző szerepe pedig elkülöníthető.

Algoritmusokat dolgoztam ki a redukálható kifejezések (úgynevezett *redexek*) detektálására és strukturált adatként (JSON) történő továbbítására a frontend felé. Ezek az adatok teszik lehetővé, hogy a webes felület vizuálisan kiemelje a soron következő redukciós lépést.

Egy ilyen vizuális reprezentáció a 4. ábrán látható, amelyet [7, Wolfram] után vettem át. A különböző kombinátoroknak más-más színeik vannak, és meg vannak különböztetve a redex-ek az argumentumaiktól.



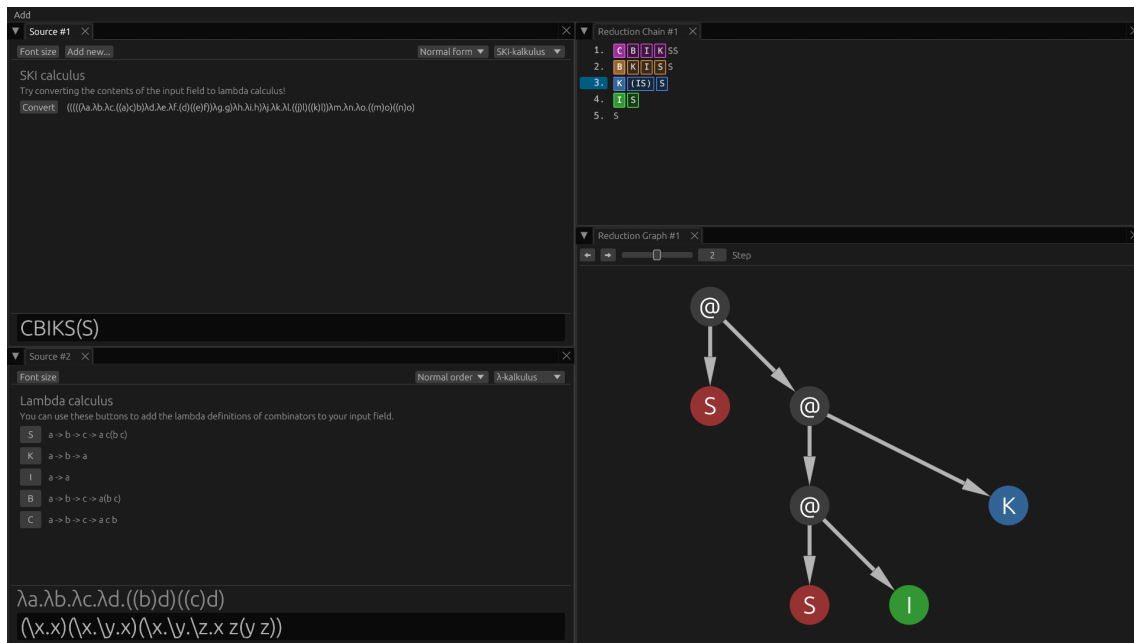
4. ábra. Egy redukciós lánc, ahol az egyes kifejezések szemantikailag vannak kiszínezve.

2.5. A frontend fejlesztése

A frontend egy teljes mértékben kliensoldalon futó alkalmazás, amelyet Rust nyelvből WebAssembly-re (WASM) fordítottam.

Az Elm-architektúrára épülő reaktív felületen a felhasználó interaktívan gépelhet be kifejezéseket, kiválaszthatja a kívánt kalkulust és a kiértékelési stratégiát. A gördülékeny felhasználói élmény érdekében a háttér folyamatokat Service Worker-ekbe (`worker.js`) szerveztem. Így a komplex és esetlegesen hosszú ideig futó redukciós fák kiértékelése nem fagyasztja le a felhasználói felületet, és a szoftver akár offline módban (PWA) is képes teljeskörűen funkcionálni. A grafikus felületet az 5. ábrán mutatom be.

Az **egui** grafikus keretrendszert használtam a felület kialakítására, amely egy úgynevezett *immediate mode* grafikus felület. Ennek a megközelítésnek az a különlegessége, hogy a felhasználói felület minden egyes képkockánál a nulláról épül fel és rajzolódik ki. A gombok, vagy beviteli mezők állapota nincsen eltárolva a memóriában két egymást követő frame között. Ez azt is jelenti, hogy a felület kirajzolása, és a felhasználói interakciók kezelése egyetlen lépésben történik.



5. ábra. A grafikus felülete a programnak. A jobb felső sarokban egy redukciós lánc látható. A bal felső sarokban egy SKI-kalkulus bemeneti ablak van, ahol megjelenik a megadott kifejezés λ -kalkulusba konvertált alakja is. A bal alsó sarokban egy λ -kalkulus bemenetű ablak helyezkedik el. Végezetül a jobb alsó sarokban a megadott SKI-kalkulus kifejezéshez tartozó redukciós gráf egyik lépése látható.

2.6. Összefoglalás

A félév során sikeresen megvalósítottam a munkatervben kitűzött célokat. Áttanulmányoztam a releváns szakirodalmat, ami alapján megterveztem az architektúrát. Kialakítottam a nyelvtanokat feldolgozó parsereket, implementáltam mind a λ -, mind a SKI-kalkulus G-machine alapú redukciós gépét, és mindezt egy WebAssembly alapú, interaktív frontendszel kötöttem össze. A Podman konténeres orákulumra épülő Property-based testing gondoskodik a motorok megbízhatóságáról. Az elkészült *Combinator Explorer* eszköz így sikeresen tölti be funkcióját: vizuális formát ad a funkcionális programozás matematikai alapjainak, hatékony és korszerű segédeszközt nyújtva a felsőoktatás számára.

3. Irodalom, és csatlakozó dokumentumok jegyzéke

3.1. A tanulmányozott irodalom jegyzéke

- [1] Arthur Charguéraud: The Locally Nameless Representation. 49. évf. 3. sz., 363–408. p. ISSN 1573-0670. URL <https://doi.org/10.1007/s10817-011-9225-2>.
- [2] H. B. Curry: Grundlagen der Kombinatorischen Logik. *American Journal of Mathematics*, 52. évf. (1930) 3. sz., 509–536. p. ISSN 0002-9327.
- [3] Flattening ASTs (and Other Compiler Data Structures). URL <https://www.cs.cornell.edu/~asampson/blog/flattening.html>.
- [4] Lambda is not a four letter word - Implementing a Functional Language with Graph Reduction. URL <https://thma.github.io/posts/2021-12-27-Implementing-a-functional-language-with-Graph-Reduction.html>.
- [5] M. Schönfinkel: Über die Bausteine der mathematischen Logik. *Mathematische Annalen*, 92. évf. (1924. szeptember) 3-4. sz., 305–316. p. ISSN 0025-5831, 1432-1807.
- [6] Raymond M. Smullyan: *To Mock a Mockingbird: And Other Logic Puzzles Including an Amazing Adventure in Combinatory Logic*. 2000, Oxford University Press. ISBN 978-0-19-280142-5.
- [7] Stephen Wolfram: Combinators: A Centennial View. *Stephen Wolfram Writings*, 2020. december.
- [8] Stephen Wolfram: Combinators and the Story of Computation. *Stephen Wolfram Writings*, 2020. december.

3.2. A csatlakozó dokumentumok jegyzéke

Az alábbi kódrészlet a `lexor-reducer/src/core/engine.rs` fájlból származik. Azt a `reduce` függvényt és `match` blokkot mutatja be, amely az S, K, I, B és C kombinátorokhoz tartozó gráf-redukciós lépéseket (az új csomópontok allokalását és a mutatók átírását) végzi el az aréna-allokátoron belül.

```

1
2   let result = match redex {
3       NodeComb::I | NodeComb::K => Node::Indirection(args[0]),
4       NodeComb::S => {
5           let xz = self.arena.insert(Node::App(args[0], args[2]));
6           let yz = self.arena.insert(Node::App(args[1], args[2]));
7           Node::App(xz, yz)
8       }
9       NodeComb::B => {
10          let yz = self.arena.insert(Node::App(args[1], args[2]));
11          Node::App(args[0], yz)
12      }
13      NodeComb::C => {
14          let xz = self.arena.insert(Node::App(args[0], args[2]));
15          Node::App(xz, args[1])
16      }
17  };

```

3. Listing. Kombinátorok redukciós szabályai a G-machine motorban